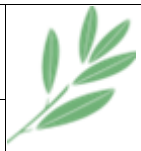
FFORDMED®	
Technology, Innovation & Affordability	

Campus Hiring Evaluation - Full Stack

Terms & Conditions

This document and the associated assessment contain confidential and proprietary information of Afford Medical Technologies Private Limited (hereinafter "**Affordmed**"). By accessing this material or participating in this assessment, you acknowledge receipt of this information for the sole purpose of evaluating your candidacy for an internship, contract, or employment with Affordmed. You hereby agree to the following:

- **Confidentiality**: You shall maintain the strict confidentiality of all information received and refrain from sharing, distributing, or disclosing any part of the information to any third party.
- **Non-Tampering**: You shall not tamper with, disrupt, or attempt to compromise any Affordmed or its vendor's cloud or software resources provided for this assessment.
- **Sole Use**: You shall use this material solely for the purpose stated herein and for no other purpose whatsoever.

Any unauthorised use, disclosure, or tampering will result in immediate disqualification from the candidacy process and may subject you to legal action. You consent to the exclusive jurisdiction of the Courts of Hyderabad/Secunderabad for any legal disputes arising from this agreement. Any reference to third parties within this document is purely coincidental and for illustrative purposes only, and does not constitute any endorsement or affiliation.

Evaluation Considerations

Time Limit: 3 Hours (No extra time for pushing to GitHub)

- It is essential to complete all the steps listed in the Pre-Test Setup document prior to working on the below test.
- You're a full stack developer working on a campus notification platform where students receive real-time updates regarding Placements, Events, and Results. You have to incrementally solve different tasks across stages. Not every stage requires coding, each stage has clear instructions on the deliverables. You're expected to commit and push your deliverables to the GitHub Repository that you created while implementing the **Logging Middleware** at frequent intervals.

Direct submission of your response at the end of the test as a single commit will result in lower points for your submission.

- As you progress through the stages, you may revise your submission for the previous stages. Your submission will be evaluated across stages both individually and cumulatively.
- At different stages of the task, there may be references to other roles within the team (like frontend developer, product manager, architect, etc.) and those are provided only as imaginary roles that they shall play. At no point should you consult or discuss your strategy or submissions with your peers. This is not a team activity.
- **Mandatory Logging Integration:** Wherever you're tasked with writing code, your implementation **MUST** extensively use the Logging Middleware you created in the Pre-Test Setup stage. Use of inbuilt language loggers or console logging is not allowed.
- **Authentication**: For the purpose of this evaluation, assume users accessing your application/code/APIs as having been pre-authorised. Your application must not require user registration or login mechanisms for access.

Deliverables

Stage 1

Assume a front-end developer colleague has asked you for REST API design, contract and structure to display notifications to the users when they are logged in. Identify the core actions that the notification platform should support. Now, you have to present the REST API endpoints along with their JSON request, response, and headers structures using an appropriate format. Define clear and consistent endpoints for each action, using predictable naming conventions, and design JSON schemas with essential fields. Also, you are to design a mechanism for real-time notifications. Submit your response as a markdown file called "notification_system_design.md" to the same repository you created while creating the logging middleware. Label your response with "Stage 1" as heading.

Stage 2

On the basis of the APIs and contract you created earlier, you now have to store the same reliably. Which persistent storage (DB) do you suggest and explain your choice. Write the applicable DB schema. What problems could arise as the data volume increases? How would you solve such problems? Write SQL or NoSQL queries based on your DB schema and the REST APIs that you designed in Stage 1. Submit your response in a new section labeled "Stage 2" by expanding the same "notification_system_design.md" file.

Stage 3

An earlier developer in the team chose a relational database for storage (MySQL or PostgreSQL or any other SQL database) about 3 months ago. Now the database has grown to 50,000 students and 5,000,000 notifications. The developer had written the below query to fetch all the unread notifications of a student as a part of the notification API that was developed, which is now performing slowly.

```
SELECT * FROM notifications
WHERE studentID = 1042 AND isRead = false
ORDER BY createdAt ASC;
```

Is this query accurate? Why is this slow? What would you change and what would be the likely computation cost? Another developer on your team suggests adding indexes on every column to be safe. Is this advice effective? Why/Why not? Write a query to find all students who got a placement notification in the last 7 days. The table contains "notificationType" as a column which accepts "notification_type" enum values. "notification_type" enum contains "Event", "Result" and "Placement". Submit your response in a new section labeled "Stage 3" by expanding the same "notification_system_design.md" file.

Stage 4

The notifications are being fetched on each page load for every student. The DB is getting overwhelmed which is causing a bad user experience. What solution will you suggest? How will you improve performance? Elaborate on the tradeoffs of each strategy you suggest. Submit your response in a new section labeled "Stage 4" by expanding the same "notification_system_design.md" file.

Stage 5

It is placement season. The HR clicks on "Notify All" and 50,000 students should get an email and an in-app notification simultaneously. Below is pseudocode for the proposed implementation.

```
function notify_all(student_ids: array, message: string):
  for student_id in student_ids:
    send_email(student_id, message)  # calls Email API
    save_to_db(student_id, message)  # DB insert
    push_to_app(student_id, message) # implementation is based on whatever real time notification mechanism you have chosen in Stage 1
```

What shortcomings do you observe with this implementation? Logs indicate that the 'send_email' call failed for 200 students midway. What now? How would you redesign

this to be reliable and fast? Should the process of saving to DB as well as sending the email happen together? Why or why not? Submit revised pseudocode along with your response to these questions in a new section labeled “Stage 5” by expanding the same “notification_system_design.md” file.

Stage 6

You’ve received user feedback from your product manager, they’d like to introduce a Priority Inbox that always displays the top ‘n’ most important unread notifications first (n could be top 10,15, 20, etc. as per user’s choice). Priority should be determined based on a combination of weight (placement > result > event) and recency. Implement your approach or solution in any language of your choice (Go, Rust, Python, TypeScript, JavaScript, Java etc). Write code only to find top 10 notifications (DB query is not expected). Your submission should be an actual functioning code file and not pseudo-code. You’re also expected to upload screenshots of your output displaying the priority notifications. Both the code and the screenshots are to be pushed to the same GitHub repository. Also note that new notifications will keep coming in. How will you maintain the top 10 efficiently? In addition to the code and screenshots, you may revise the same “notification_system_design.md” file to also explain your approach in this stage in a new section labeled “Stage 6”.

To simplify your task, you’re also provided with the below Notification API. You are expected to use the API to fetch the notifications. You need not store them in a database, nor are you supposed to hard-code or create notifications yourself.

Notification API (GET)

```
http://4.224.186.213/evaluation-service/notifications
```

Constraints

- API is a protected Route

Response (Status Code: 200)

```
{
  "notifications": [
    {
      "ID": "d146095a-0d86-4a34-9e69-3900a14576bc",
      "Type": "Result",
      "Message": "mid-sem",
      "Timestamp": "2026-04-22 17:51:30"
    },
    {
```

```
    "ID": "b283218f-ea5a-4b7c-93a9-1f2f240d64b0",
    "Type": "Placement",
    "Message": "CSX Corporation hiring",
    "Timestamp": "2026-04-22 17:51:18"
  },
  {
    "ID": "81589ada-0ad3-4f77-9554-f52fb558e09d",
    "Type": "Event",
    "Message": "farewell",
    "Timestamp": "2026-04-22 17:51:06"
  },
  {
    "ID": "0005513a-142b-4bbc-8678-eefec65e1ede",
    "Type": "Result",
    "Message": "mid-sem",
    "Timestamp": "2026-04-22 17:50:54"
  },
  {
    "ID": "ea836726-c25e-4f21-a72f-544a6af8a37f",
    "Type": "Result",
    "Message": "project-review",
    "Timestamp": "2026-04-22 17:50:42"
  },
  {
    "ID": "003cb427-8fc6-47f7-bb00-be228f6b0d2c",
    "Type": "Result",
    "Message": "external",
    "Timestamp": "2026-04-22 17:50:30"
  },
  {
    "ID": "e5c4ff20-31bf-4d40-8f02-72fda59e8918",
    "Type": "Result",
    "Message": "project-review",
    "Timestamp": "2026-04-22 17:50:18"
  },
  {
    "ID": "1cfce5ee-ad37-4894-8946-d707627176a5",
    "Type": "Event",
    "Message": "tech-fest",
    "Timestamp": "2026-04-22 17:50:06"
```

```
    },
    {
      "ID": "cf2885a6-45ac-4ba0-b548-6e9e9d4c52c8",
      "Type": "Result",
      "Message": "project-review",
      "Timestamp": "2026-04-22 17:49:54"
    },
    {
      "ID": "8a7412bd-6065-4d09-8501-a37f11cc848b",
      "Type": "Placement",
      "Message": "Advanced Micro Devices Inc. hiring",
      "Timestamp": "2026-04-22 17:49:42"
    }
  ]
}
```


Stage 7

You've received a go-ahead from your software architect to implement the frontend for the application. You are to develop a responsive React/Next (Use of other non-React based frameworks is not permitted) application that displays all Notifications as well as priority Notifications (enabling display of limited top "n" notifications as well as filter on notification type) on a different page. You're expected to distinguish between new and already viewed notifications by relying on suitable frontend implementation. Submit your response as a new sub-directory within the same Github repository along with a video recording of your application's pages and functionality (both desktop and mobile views). Your React/Next application must run exclusively on http://localhost:3000. Care must be taken to avoid cluttering the page. The UI must prioritise user experience, with a focus on highlighting key elements of each page. Your application must adhere to all specified API and frontend requirements and general constraints, demonstrating robust error handling, high code quality, and efficient API/UI design suitable for a production environment. Use Material UI only for styling. If you are not familiar with Material UI, use native CSS. Use of ShadCN or other CSS Libraries is prohibited. Solely relying on native CSS or not using Material UI will result in lower scores.

To ease your implementation, the notifications API http://4.224.186.213/evaluation-service/notifications has now been expanded to include the below query parameters

Query Parameters

- "limit"
- "page"

- “notification_type”

Notification Types Supported

```
"Event"
"Result"
"Placement"
```

Afford Medical Technologies Private Limited

B 230 2nd Main Road, Sainikpuri, Hyderabad-500094, Telangana, INDIA. Phone: 91-40-27117068/27116133,

Web: www.affordmed.com, E-mail: contact@affordmed.com, CIN: U72200TG2007PTC056067, URN: UDYAM-TS-20-0013532